Reporte

EQUIPO 13	
Nombre: Edgar Jesus Del Angel Martinez Israel Alejandro Pantoja Esquivel Andrea Fernanda Rodríguez Niño	**Matrícula:** 2160745 2155971 2160663
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Módulo NetAcad: 4	**Actividad:** Laboratorio guiado. Programa para administrar pedidos de una tienda ficticia TechStore.
Fecha: 28/10/2026	

Bibliografía (formato APA)

- Beck, K. (2002). Test-driven development: By example. Addison-Wesley Professional.
- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Millett, S., & Tune, N. (2015). Patterns, principles, and practices of domain-driven design. Wrox.
- Vernon, V. (2013). Implementing domain-driven design. Addison-Wesley Professional.

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

Procedimiento:

Diagrama de estados

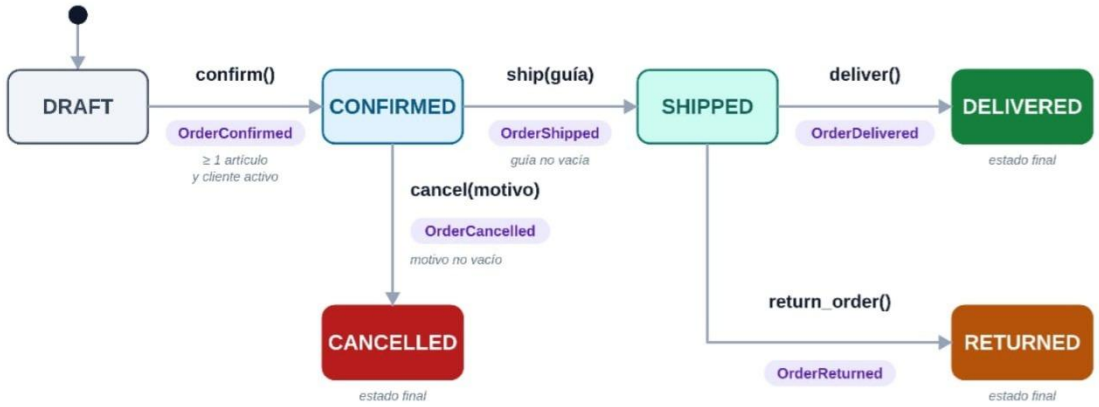


Figura 1. Máquina de estados y Domain Events de PetMarket.

El pedido inicia en DRAFT. La confirmación lo lleva a CONFIRMED cuando contiene al menos un artículo y el cliente está activo. Desde CONFIRMED puede enviarse con una guía no vacía o cancelarse con un motivo. Un pedido enviado puede entregarse o devolverse. DELIVERED, RETURNED y CANCELLED son estados finales.

Transiciones permitidas y precondiciones

Estado inicial	Operación	Estado final	Domain Event	Precondición
DRAFT	confirm()	CONFIRMED	OrderConfirmed	Al menos un artículo y cliente activo.
CONFIRMED	ship(guía)	SHIPPED	OrderShipped	Guía no vacía.
CONFIRMED	cancel(motivo)	CANCELLED	OrderCancelled	Motivo no vacío.
SHIPPED	deliver()	DELIVERED	OrderDelivered	Pedido en SHIPPED.
SHIPPED	return_order()	RETURNED	OrderReturned	Pedido en SHIPPED.

Ejemplos de transiciones prohibidas: confirmar un pedido vacío o con cliente inactivo; enviar sin guía; cancelar después de que el pedido fue enviado; entregar o devolver un pedido que no está en SHIPPED; y ejecutar una transición sobre un estado final.

Estructura de un Domain Event

En PetMarket, los eventos identifican hechos del negocio que ya ocurrieron. El nombre se expresa en pasado, por ejemplo OrderConfirmed u OrderShipped. De acuerdo con la

implementación del proyecto, los eventos se conservan en la colección de eventos del pedido y se agregan después de que la transición cambia el estado.

Estructura conceptual del evento:

DomainEvent(
name: str, # nombre del hecho ocurrido
order_id: str, # identificador del pedido
occurred_at: datetime # fecha y hora del registro
)

El nombre del evento describe un hecho concluido, no una solicitud pendiente.

El evento está asociado al pedido que realizó la transición.

Los eventos se agregan únicamente después de validar y completar una operación.

Una operación rechazada conserva el estado del pedido y no añade el evento correspondiente.

Los cinco eventos, uno por uno

1. OrderConfirmed

Hecho que representa: el pedido pasó de DRAFT a CONFIRMED.

Operación que lo genera: Order.confirm() (API: POST /orders/{order_id}/confirm).

Condiciones para registrarlo: el pedido debe estar en DRAFT, contener al menos un artículo y pertenecer a un cliente activo.

Cuando no se registra: si el pedido está vacío o el cliente está inactivo, la confirmación se rechaza y no se agrega OrderConfirmed.

2. OrderShipped

Hecho que representa: el pedido fue enviado y quedó en SHIPPED.

Operación que lo genera: Order.ship(tracking_number) (API: POST /orders/{order_id}/ship).

Condiciones para registrarlo: el pedido debe estar en CONFIRMED y la guía debe contener un valor no vacío.

Cuando no se registra: un intento de envío sin guía se rechaza; el pedido permanece CONFIRMED y no se registra OrderShipped.

3. OrderCancelled

Hecho que representa: el pedido fue cancelado y quedó en CANCELLED.

Operación que lo genera: Order.cancel(reason) (API: POST /orders/{order_id}/cancel).

Condiciones para registrarlo: el pedido debe estar en CONFIRMED y el motivo de cancelación no puede estar vacío.

Cuando no se registra: si el pedido ya está SHIPPED, la cancelación no está permitida. Tampoco se registra el evento si falta el motivo.

4. OrderDelivered

Hecho que representa: el pedido enviado fue entregado y quedó en DELIVERED.

Operación que lo genera: Order.deliver() (API: POST /orders/{order_id}/deliver).

Condiciones para registrarlo: el pedido debe estar en SHIPPED.

Ejemplo: al completar confirmar → enviar → entregar, se registran OrderConfirmed, OrderShipped y OrderDelivered.

5. OrderReturned

Hecho que representa: el pedido enviado fue devuelto y quedó en RETURNED.

Operación que lo genera: Order.return_order() (API: POST /orders/{order_id}/return).

Condiciones para registrarlo: el pedido debe estar en SHIPPED.

Ejemplo: al completar confirmar → enviar → devolver, se registran OrderConfirmed, OrderShipped y OrderReturned.

Resumen: evento, transición y comportamiento ante un rechazo

Evento	Transición	Cuándo se registra	Si la operación falla
OrderConfirmed	DRAFT → CONFIRMED	Artículo(s) y cliente activo.	Sin cambio de estado ni evento.
OrderShipped	CONFIRMED → SHIPPED	Guía no vacía.	Permanece CONFIRMED; no OrderShipped.
OrderCancelled	CONFIRMED → CANCELLED	Motivo no vacío.	Permanece en estado actual; no OrderCancelled.
OrderDelivered	SHIPPED → DELIVERED	Entrega válida desde SHIPPED.	Sin transición ni OrderDelivered.
OrderReturned	SHIPPED → RETURNED	Devolución válida desde SHIPPED.	Sin transición ni OrderReturned.

Momento en que se registran

Las operaciones siguen este orden lógico:

Validar el estado actual y las precondiciones de la operación.

Actualizar el estado del pedido cuando todas las validaciones se cumplen.

Registrar el Domain Event que describe la transición completada.

El evento no se registra antes de las validaciones. Así se evita que el sistema comunique un envío, una cancelación o una entrega que en realidad no ocurrió. Si una operación es rechazada, el estado permanece igual y la colección de eventos no aumenta.

¿Por qué solo después de una transición válida?

Un Domain Event representa un hecho del negocio que ya sucedió. Por ejemplo, OrderShipped significa que el pedido efectivamente salió con una guía válida. Si se agregara el evento antes de verificar el estado y los datos requeridos, podrían quedar registros contradictorios: el evento indicaría que se envió el pedido, aunque el sistema hubiera rechazado la operación. Registrar el evento al final de la transición mantiene coherentes el estado del pedido y su historial.